

ЛАБОРАТОРНАЯ РАБОТА 1

Тема: «Разработка и тестирование RESTful API-сервиса с архитектурой MVC на Node.js»

Цель: Научиться проектировать слой маршрутов (Routing), контроллеров (Controller) и бизнес-логики (Service/Model) без использования баз данных (работаем с In-memory хранилищем).

Шаг 0. Подготовка окружения

1. Установите **VS Code** (бесплатно).
 2. Установите **Node.js** (LTS-версию) с официального сайта.
 3. Создайте папку lab1_arch и откройте её в VS Code.
 4. Откройте **терминал** в VS Code (Ctrl + Shift + ~).
-

Шаг 1. Инициализация проекта (package.json)

В терминале выполните команду инициализации. На все вопросы жмите Enter (по умолчанию):

```
bash
npm init -y
```

Шаг 2. Установка зависимостей

Установим фреймворк для сервера, парсер JSON, и библиотеки для тестов:

```
bash
npm install express
npm install --save-dev jest supertest nodemon
```

Что мы поставили:

- **express** — сам сервер.
- **jest** — фреймворк для модульного тестирования (от Facebook).
- **supertest** — библиотека для HTTP-запросов в тестах (позволяет тестировать API без реального запуска порта).
- **nodemon** — для автоперезапуска сервера при изменениях (ставим в разработку).

Шаг 3. Создаем структуру папок (MVC)

В корневой папке создайте вручную следующую иерархию:

```
text
lab1_arch
├─ src
│  ├─ controllers
│  │   └─ userController.js
│  ├─ services
│  │   └─ userService.js
│  └─ routes
│       └─ userRoutes.js
└─ app.js
├─ tests
│   └─ api.test.js
└─ server.js (корневой файл запуска)
```

Шаг 4. Пишем «Service» (слой бизнес-логики и In-memory БД)

Этот слой отвечает за хранение данных и операции над ними. Откройте `src/services/userService.js` и вставьте:

```
javascript
// In-мемори хранилище (имитация базы данных)
let users = [
  { id: 1, name: 'Alice', email: 'alice@example.com' },
  { id: 2, name: 'Bob', email: 'bob@example.com' }
];
let nextId = 3;

// CRUD операции
const getAllUsers = () => users;

const getUserById = (id) => users.find(user => user.id === id);

const createUser = (name, email) => {
  const newUser = { id: nextId++, name, email };
  users.push(newUser);
  return newUser;
}
```

```

};

const updateUser = (id, name, email) => {
  const user = getUserById(id);
  if (!user) return null;
  if (name) user.name = name;
  if (email) user.email = email;
  return user;
};

const deleteUser = (id) => {
  const index = users.findIndex(user => user.id === id);
  if (index === -1) return false;
  users.splice(index, 1);
  return true;
};

module.exports = {
  getAllUsers,
  getUserById,
  createUser,
  updateUser,
  deleteUser
};

```

Шаг 5. Пишем «Controller» (слой обработки запросов и ответов)

Контроллер принимает req и res, вызывает сервис и отправляет HTTP-ответ. Откройте src/controllers/userController.js:

```

javascript
const userService = require('../services/userService');

// Получить всех
const getUsers = (req, res) => {
  const users = userService.getAllUsers();
  res.status(200).json(users);
};

// Получить одного по ID
const getUser = (req, res) => {
  const id = parseInt(req.params.id);

```

```

const user = userService.getUserById(id);
if (!user) {
  return res.status(404).json({ error: 'User not found' });
}
res.status(200).json(user);
};

// Создать пользователя
const createUser = (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ error: 'Name and email are required' });
  }
  const newUser = userService.createUser(name, email);
  res.status(201).json(newUser);
};

// Обновить пользователя
const updateUser = (req, res) => {
  const id = parseInt(req.params.id);
  const { name, email } = req.body;
  const updatedUser = userService.updateUser(id, name, email);
  if (!updatedUser) {
    return res.status(404).json({ error: 'User not found' });
  }
  res.status(200).json(updatedUser);
};

// Удалить пользователя
const deleteUser = (req, res) => {
  const id = parseInt(req.params.id);
  const deleted = userService.deleteUser(id);
  if (!deleted) {
    return res.status(404).json({ error: 'User not found' });
  }
  res.status(204).send(); // No Content
};

module.exports = {
  getUsers,
  getUser,
  createUser,
  updateUser,
  deleteUser
};

```

```
};
```

Шаг 6. Пишем «Routes» (слой маршрутизации)

Откройте `src/routes/userRoutes.js`. Здесь мы связываем URL с методами контроллера:

```
javascript
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

// Определяем REST-маршруты
router.get('/', userController.getUsers);    // GET /api/users
router.get('/:id', userController.getUser);  // GET /api/users/1
router.post('/', userController.createUser);  // POST /api/users
router.put('/:id', userController.updateUser); // PUT /api/users/1
router.delete('/:id', userController.deleteUser); // DELETE /api/users/1

module.exports = router;
```

Шаг 7. Собираем приложение (app.js)

Это сердце вашего приложения. Здесь подключаются мидлвары и роуты. Откройте `src/app.js`:

```
javascript
const express = require('express');
const userRoutes = require('./routes/userRoutes');

const app = express();

// Встроенный парсер JSON (обязательно)
app.use(express.json());

// Подключаем роуты с префиксом /api/users
app.use('/api/users', userRoutes);

// Обработка 404 для несуществующих маршрутов (архитектурный момент)
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});
```

```
});
```

```
module.exports = app;
```

Шаг 8. Создаем точку входа (server.js)

Этот файл нужен только для запуска сервера на порту. Откройте корневой `server.js`:

```
javascript
```

```
const app = require('./src/app');
```

```
const PORT = process.env.PORT || 3000;
```

```
app.listen(PORT, () => {  
  console.log(`Server is running on http://localhost:${PORT}`);  
});
```

Шаг 9. Настройка скриптов в package.json

Откройте `package.json`. В раздел `"scripts"` добавьте три строчки (замените то, что там уже есть, на это):

```
json
```

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js",  
  "test": "jest"  
}
```

Шаг 10. Пишем интеграционные тесты (Jest + Supertest)

Это самый важный архитектурный шаг — тестируем API без запуска сервера. Откройте `tests/api.test.js`:

```
javascript
```

```
const request = require('supertest');
```

```
const app = require('../src/app');
```

```

describe('User API Integration Tests', () => {

  // Тест GET /api/users (получение всех)
  test('GET /api/users should return array of users', async () => {
    const response = await request(app).get('/api/users');
    expect(response.statusCode).toBe(200);
    expect(Array.isArray(response.body)).toBe(true);
    expect(response.body.length).toBe(2); // Начальные Алиса и Боб
  });

  // Тест GET /api/users/:id (получение одного)
  test('GET /api/users/1 should return Alice', async () => {
    const response = await request(app).get('/api/users/1');
    expect(response.statusCode).toBe(200);
    expect(response.body).toHaveProperty('name', 'Alice');
    expect(response.body).toHaveProperty('email', 'alice@example.com');
  });

  // Тест POST /api/users (создание)
  test('POST /api/users should create new user', async () => {
    const newUser = { name: 'Charlie', email: 'charlie@example.com' };
    const response = await request(app)
      .post('/api/users')
      .send(newUser);
    expect(response.statusCode).toBe(201);
    expect(response.body).toHaveProperty('id', 3);
    expect(response.body.name).toBe('Charlie');
  });

  // Тест PUT /api/users/:id (обновление)
  test('PUT /api/users/2 should update Bob', async () => {
    const updatedData = { name: 'Robert', email: 'robert@example.com' };
    const response = await request(app)
      .put('/api/users/2')
      .send(updatedData);
    expect(response.statusCode).toBe(200);
    expect(response.body.name).toBe('Robert');
  });

  // Тест DELETE /api/users/:id (удаление)
  test('DELETE /api/users/1 should delete Alice', async () => {
    const response = await request(app).delete('/api/users/1');
    expect(response.statusCode).toBe(204);
  });
});

```

```

// Проверим, что пользователь действительно удален
const check = await request(app).get('/api/users/1');
expect(check.statusCode).toBe(404);
});

// Тест на валидацию (400 ошибка)
test('POST /api/users without email should return 400', async () => {
  const invalidUser = { name: 'Invalid' };
  const response = await request(app)
    .post('/api/users')
    .send(invalidUser);
  expect(response.statusCode).toBe(400);
  expect(response.body).toHaveProperty('error');
});
});

```

Шаг 11. Запуск и проверка (пошагово)

1. Запустите сервер в режиме разработки (авто-перезагрузка):
В терминале выполните: `npm run dev`
Вы увидите: `Server is running on http://localhost:3000`
 2. Проверка через браузер или curl:
 - Откройте браузер и перейдите на `http://localhost:3000/api/users` — увидите JSON с Алисой и Бобом.
 3. Запуск тестов (отдельный терминал или остановите сервер `Ctrl+C`):
В терминале выполните: `npm test`
Вы должны увидеть зеленый отчет о прохождении VCEX 6 тестов (PASS).
-

Шаг 12. Оформление отчета

В вашем отчете к лабораторной работе должны быть:

1. ER-диаграмма сущности User (поле id, name, email).
2. Схема архитектуры MVC с указанием потока данных:
`Client (HTTP) → Routes → Controller → Service (In-memory) → Response`.
3. Описание REST-принципов: какие методы (GET, POST, PUT, DELETE) для каких ресурсов используются и коды ответов (200, 201, 204, 400, 404).
4. Скриншоты успешного прохождения тестов (`npm test`) в терминале.
5. Скриншот выполнения `GET /api/users` через браузер или Postman (можно бесплатно установить).

6. **Вывод:** какие архитектурные слои удалось разделить и почему тестирование через Supertest лучше, чем ручное тестирование через Postman (автоматизация, регрессия, CI/CD).